

S3: Refine Document Embedding

Discussion

- Are the remaining words (after dropping low-df and stop-words) equally important?
- Which words are important role in determining the information in a document or discriminating document?
 - E.g., Unstructured vs. Data
- Appearing in more documents or less?
- Consider a term in a document that is rare in the collection
 - This term captures the uniqueness of the document, so it more distincts the document from those doesn't contain it.
- Shall we give **equal-to-others or more** weights to those important terms?
 - We want a higher weights for more important terms, so the distance between this document and the rest will be longer

Word weighting by TF-IDF

- The most popular weighting scheme:
 - $TF(t, d)$: (Normalized) Term Frequency (number of term t occurrences in one document d), similar to term-document frequency
 - $DF(t, D)$: (Normalized) Document Frequency (number of documents in collection D containing the term t) -> **reweight**
 - TF-IDF: Term Frequency – Inversed Document Frequency

$$TF - IDF(t, d, D) = TF(t, d) \cdot \log \frac{N}{DF(t, D)}$$

- TF-IDF value of a term in a document
 - Increases with the number of occurrences within the document
 - Increases with the rarity of the term in the collection

Python: TF-IDF

- Use a TF-IDF to replace term frequency in the term-document matrix for the bag-of-words representation

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer2 = TfidfVectorizer()
vectorizer2.fit(c1)
print(vectorizer2.vocabulary_)

v2 = vectorizer2.transform(c1)
print(v2.toarray())

c2 = ["hello world", "jinyang is calling"]
v_c2 = vectorizer2.transform(c2)
print(v_c2.toarray())
```

Another issue: loss of order information

- Jinyang is **a good guy**, he is **not bad**
- Jinyang is **not a good guy**, he **is bad**
- Are they the same?
- What is the distance between them in the above-mentioned embedding?
- How to preserve (some) order information?

Preserve phrase level information

- Phrase level information: **local order information**
- We can also include token pairs, triplets, etc. in representations
 - 1-grams for words, tokens
 - 2-grams for two-words-phrase, token pairs
 - n-grams for a n-words phrase
- Bag of words -> Bag of **n-grams**
 - Jinyang **is a** good guy, he is **not bad**
 - Jinyang **is not** a good guy, he **is bad**

Python: bag of n-grams

- http://scikit-learn.org/stable/modules/feature_extraction.html

```
vectorizer3 = TfidfVectorizer(ngram_range=(1, 2), min_df=2)
vectorizer3.fit(c1)
v3 = vectorizer3.transform(c1)

print(v3.toarray())
print(vectorizer3.vocabulary_)
```

Alert: high dimensional again

- Google n-gram corpus:
- <http://googleresearch.blogspot.com/2006/08/all-our-n-gram-are-belong-to-you.html#links>
 - File sizes: approx. 24 GB compressed (gzip'ed) text files
 - Number of sentences: 95,119,665,584
 - Number of unigrams: 13,588,391
 - Number of bigrams: 314,843,401
 - Number of trigrams: 977,069,902
 - Number of fourgrams: 1,313,818,354
 - Number of fivegrams: 1,176,470,663

Preserve part-of-speech structure information

- I **drink** water in parties
- I grab a **drink** in parties
- **Part-of-speech level** order information matters!
 - Preserve part-of-speech structure information before BOW (BOngrams) **vectorization**
- **POS Tag**: tag word-types enabling to differentiate words functions and to preserve **part-of-speech (POS) level** information

Tag classes

- Classify words that somehow 'behave' alike:
 - Appear in similar contexts
 - Perform similar functions in sentences
 - Undergo similar transformations
- 8 conventional word classes of parts of speech

part of speech	function or "job"	example words	example sentences
<u>Verb</u>	action or state	(to) be, have, do, like, work, sing, can, must	EnglishClub.com is a web site. I like EnglishClub.com.
<u>Noun</u>	thing or person	pen, dog, work, music, town, London, teacher, John	This is my dog . He lives in my house . We live in London .
<u>Adjective</u>	describes a noun	a/an, the, 69, some, good, big, red, well, interesting	My dog is big . I like big dogs.
<u>Adverb</u>	describes a verb, adjective or adverb	quickly, silently, well, badly, very, really	My dog eats quickly . When he is very hungry, he eats really quickly.
<u>Pronoun</u>	replaces a noun	I, you, he, she, some	Tara is Indian. She is beautiful.
<u>Preposition</u>	links a noun to another word	to, at, after, on, but	We went to school on Monday.
<u>Conjunction</u>	joins clauses or sentences or words	and, but, when	I like dogs and I like cats. I like cats and dogs. I like dogs but I don't like cats.
<u>Interjection</u>	short exclamation, sometimes inserted into a sentence	oh!, ouch!, hi!, well	Ouch! That hurts! Hi! How are you? Well , I don't know.

http://www.englishclub.com/grammar/parts-of-speech_1.htm

POS Tag examples

verb
Stop!

noun	verb
John	works.

noun	verb	verb
John	is	working.

pronoun	verb	noun
She	loves	animals.

noun	verb	adjective	noun
Animals	like	kind	people.

noun	verb	noun	adverb
Tara	speaks	English	well.

noun	verb	adjective	noun
Tara	speaks	good	English.

pronoun	verb	preposition	adjective	noun	adverb
She	ran	to	the	station	quickly.

pron.	verb	adj.	noun	conjunction	pron.	verb	pron.
She	likes	big	snakes	but	I	hate	them.

Here is a sentence that contains every part of speech:

interjection	pron.	conj.	adj.	noun	verb	prep.	noun	adverb
Well,	she	and	young	John	walk	to	school	slowly.

- Not good enough, too crude

http://www.englishclub.com/grammar/parts-of-speech_1.htm

Finer classes

- Closed class: relatively fixed and small set:
 - Prepositions: of, in, by, ...
 - Auxiliaries: may, can, will, had, been, ...
 - Pronouns: I, you, she, mine, his, them, ...
 - Usually function words (short common words which play a role in grammar)
- Open class: productive and large set
 - Nouns
 - Verbs
 - Adjectives
 - Adverbs

Open class words: nouns and verbs

- Open class words: nouns
 - Proper nouns
 - Simon Business School, Rochester, NY, U.S.A
 - English capitalizes these
 - Many have abbreviations
 - Common nouns
 - All the rest
 - Count nouns vs. mass nouns
 - Count: Have plurals, countable: goat/goats, one goat, two goats
 - Mass: Not countable: salt, communism
- Open class words: verbs
 - Represent actions (**walk, ate**), processes (**provide, see**), and states (**be, seem**)
 - Morphological affixes (**eat/eats/eaten**)
 - Many subclasses, e.g.,
 - $\text{eats/V} \Rightarrow \text{eat/VB, eat/VBP, eats/VBZ, ate/VBD, eaten/VBN, eating/VBG, ...}$
 - Reflect morphological form & syntactic function
 - Normalization?

Open class words: adjectives and adverbs

- Adjectives: identify properties or qualities of nouns
 - Color, size, age, ...
 - Comparative, superlative
- Adverbs: also modify things (verbs, adjectives, adverbs)
 - E.g., the very happy man walked home extremely slowly yesterday.
 - Comparative, superlative
 - Functions
 - Directional/locative adverbs (here, home, downhill)
 - Degree adverbs (extremely, very, somewhat)
 - Manner adverbs (slowly, slinkily, delicately)
 - Temporal adverbs (Monday, tomorrow)

Penn Treebank tag set

Tag	Description	Example	Tag	Description	Example
CC	Coordin. Conjunction	<i>and, but, or</i>	SYM	Symbol	<i>+, %, &</i>
CD	Cardinal number	<i>one, two, three</i>	TO	“to”	<i>to</i>
DT	Determiner	<i>a, the</i>	UH	Interjection	<i>ah, oops</i>
EX	Existential ‘there’	<i>there</i>	VB	Verb, base form	<i>eat</i>
FW	Foreign word	<i>mea culpa</i>	VBD	Verb, past tense	<i>ate</i>
IN	Preposition/sub-conj	<i>of, in, by</i>	VBG	Verb, gerund	<i>eating</i>
JJ	Adjective	<i>yellow</i>	VCN	Verb, past participle	<i>eaten</i>
JJR	Adj., comparative	<i>bigger</i>	VBP	Verb, non-3sg pres	<i>eat</i>
JJS	Adj., superlative	<i>wildest</i>	VBZ	Verb, 3sg pres	<i>eats</i>
LS	List item marker	<i>1, 2, One</i>	WDT	Wh-determiner	<i>which, that</i>
MD	Modal	<i>can, should</i>	WP	Wh-pronoun	<i>what, who</i>
NN	Noun, sing. or mass	<i>llama</i>	WP\$	Possessive wh-	<i>whose</i>
NNS	Noun, plural	<i>llamas</i>	WRB	Wh-adverb	<i>how, where</i>
NNP	Proper noun, singular	<i>IBM</i>	\$	Dollar sign	<i>\$</i>
NNPS	Proper noun, plural	<i>Carolinas</i>	#	Pound sign	<i>#</i>
PDT	Predeterminer	<i>all, both</i>	“	Left quote	<i>(‘ or “)</i>
POS	Possessive ending	<i>’s</i>	”	Right quote	<i>(’ or ”)</i>
PRP	Personal pronoun	<i>I, you, he</i>	(	Left parenthesis	<i>([, ({ , <)</i>
PRP\$	Possessive pronoun	<i>your, one’s</i>	)	Right parenthesis	<i>([, { , >)</i>
RB	Adverb	<i>quickly, never</i>	,	Comma	<i>,</i>
RBR	Adverb, comparative	<i>faster</i>	.	Sentence-final punc	<i>(. ! ?)</i>
RBS	Adverb, superlative	<i>fastest</i>	:	Mid-sentence punc	<i>(: ; ... - -)</i>
RP	Particle	<i>up, off</i>			

- IBM Corp. and Alvarion Inc. have established an alliance to offer wireless systems to municipalities and their public safety agencies, Alvarion announced.
- IBM/NNP Corp./NNP and/CC Alvarion/NNP Inc./NNP have/VBP established/VBN an/DT alliance/NN to/TO offer/VB wireless/JJ systems/NNS to/TO municipalities/NNS and/CC their/PRP\$ public/JJ safety/NN agencies/NNS ,/, Alvarion/NNP announced/VBD ./.
- I/PRP **drink**/VBP water/NN in/IN parties/NNS
- I/PRP grab/VBP a/DT **drink**/NN in/IN parties/NNS

Discussion

- Question: would you do words normalization before or after POS tagging?
- Question: would POS tagging increase dimensions of the BOW?

Python: POS tag

```
##POS TAG
d4 = "I drink water in parties"
d5 = "I grab a drink in parties"
token4 = nltk.word_tokenize(d4)

POS_token4 = nltk.pos_tag(token4)

c2 = [d4, d5]
POS_c2 = []
for doc in c2:
    token_doc = nltk.word_tokenize(doc)
    POS_token_doc = nltk.pos_tag(token_doc)
    POS_token_temp = []
    for i in POS_token_doc:
        POS_token_temp.append(i[0] + i[1])
    POS_c2.append(" ".join(POS_token_temp))

vectorizer4 = TfidfVectorizer()
vectorizer4.fit(POS_c2)
print(vectorizer4.vocabulary_)

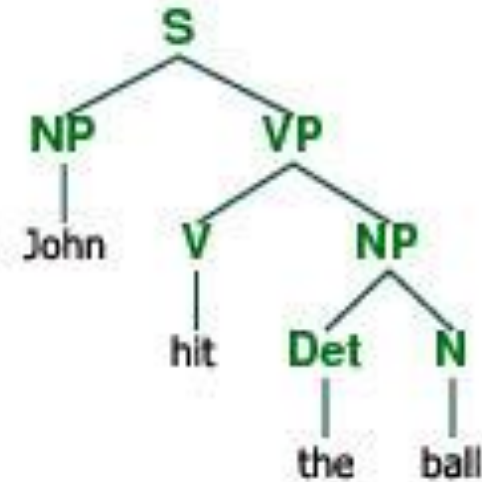
POS_v3 = vectorizer4.transform(POS_c2)
print(POS_v3.toarray())
```

Full order information

- For some tasks, **order** matters more!
 - Output will also consider order information
 - Translations
 - Voice recognition
- Example
 - Jack loves Ross **v.s** Ross loves Jack
 - Jack/NNP loves/VBZ Ross/NNP **v.s** Ross/NNP loves/VBZ Jack/NNP
- Are they the same after POS tagging and BOW representation?

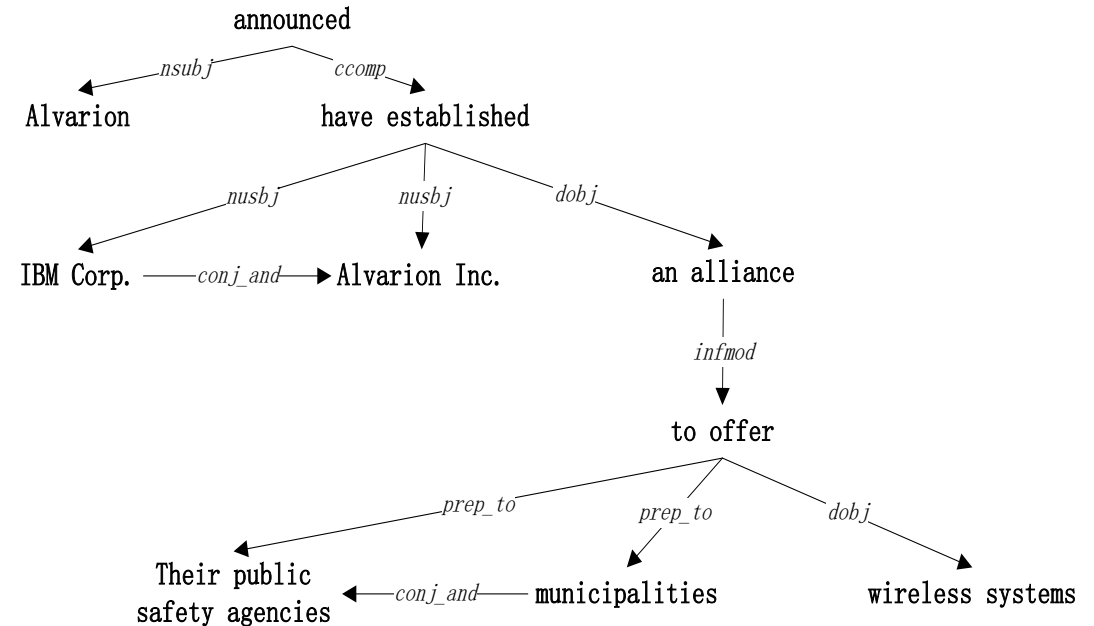
Parse tree

- A further improvement of POS tag
- **Parsing** provides maximum structural information per sentence
 - Fully keeps the order
- Input: a sentence
- Output: a parse tree



Example

- IBM/NNP Corp./NNP and/CC Alvarion/NNP Inc./NNP have/VBP established/VBN an/DT alliance/NN to/TO offer/VB wireless/JJ systems/NNS to/TO municipalities/NNS and/CC their/PRP\$ public/JJ safety/NN agencies/NNS ,/, Alvarion/NNP announced/VBD ./.



Limitation

- **Tree structure** (high dimensional as well) rather than a vector
- **No more BOngram**, not compatible with classic models
- Costly to obtain, recursive algorithm (see appendix)
- Difficult to use anywhere, e.g., neural networks
- Question: can we use BOngram to fully preserve document-level order information?

Mini Group Project 1a

There are 100 reviews for restaurants and films in a collection under the attached csv file on Blackboard. Each review is treated as a text document. In this assignment, you are required to process those documents such that each of them will be finally numerically represented. Please follow the steps listed below:

1. Tokenize each review in the collection.
2. Lemmatize all the words, convert in lowercase.
3. Remove all the stop-words and the punctuations.
4. Based on the output in step 3, convert each of the reviews in a TD-IDF vector. The minimal document frequency for each term is 3. Also, include 2-gram.

Please submit:

- A python code with your comment # reporting the dimensionality of your final output.
- A CSV file saves your final TF-IDF matrix (step 4). Each review corresponds to one row, each column corresponds to one term.

Some coding helps

- You can integrate the nltk tools into sklearn's vectorizers

```
from sklearn.feature_extraction.text import CountVectorizer
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

stemmer = PorterStemmer()

def stem_tokenizer(text):
    tokens = word_tokenize(text)
    return [stemmer.stem(t) for t in tokens]

vectorizer = CountVectorizer(tokenizer=stem_tokenizer, stop_words='english')
```




Appendix: Part-of-Speech Tagging Algorithm

Training data (Annotated text)

<i>This</i>	<i>sentence</i>	<i>serves</i>	<i>as</i>	<i>an</i>	<i>example</i>	<i>of</i>	<i>annotated</i>	<i>text...</i>
Det	N	V1	P	Det	N	P	V2	N

"This is a new sentence." → **POS Tagger** → *This is a new sentence.*
Det Aux Det Adj N

Pick the **most likely** tag sequence.

$$p(w_1, \dots, w_k, t_1, \dots, t_k) = \begin{cases} p(t_1 | w_1) \dots p(t_k | w_k) p(w_1) \dots p(w_k) \\ \prod_{i=1}^k p(w_i | t_i) p(t_i | t_{i-1}) \end{cases}$$

Independent assignment
Most common tag

Partial dependency
(HMM)

Appendix: Parse Tree Algorithm

Choose **most likely** parse tree

Probabilistic CFG	
Grammar	
$S \rightarrow NP VP$	1.0
$NP \rightarrow Det BNP$	0.3
$NP \rightarrow BNP$	0.4
$NP \rightarrow NP PP$	0.3
$BNP \rightarrow N$	...
$VP \rightarrow V$	...
$VP \rightarrow Aux V NP$	...
$VP \rightarrow VP PP$	...
$PP \rightarrow P NP$	1.0
Lexicon	
$V \rightarrow chasing$	0.01
$Aux \rightarrow is$	...
$N \rightarrow dog$	0.003
$N \rightarrow boy$	...
$N \rightarrow playground$	...
$Det \rightarrow the$	...
$Det \rightarrow a$	...
$P \rightarrow on$	...

